CS224

Lab 4

Section 4

Yusuf Yiğit Korkmaz

22201749

16.03.2025

b:

Location (hex)	Machine Instruction (hex)	Assembly Code
00	20020005	addi $v0, $zero,5
04	2003000C	addi $v1, $zero, 12
08	206FFF7	addi $a3, $v1, -3
0C	00E22025	or $a0, $a3, $v0
10	00642824	and $a1, $v1, $a0
14	00A42820	add $a1, $a1, $a0
18	10A7000A	beq $a1, $a1, $a0
1C	0064202A	slt $a0, $v1, $a0
20	10800001	beq $a0, $0, 1
24	20050000	addi $a1, $zero, 0
28	00E2202A	slt $a0, $a3, $v0
2C	00853820	add $a3, $a0, $a1
30	00E23822	sub $a3, $a3, $v0
34	AC670044	sw $a3, 0x0044, $v1
38	8C020050	lw $v0, 80($zero)
3C	08000011	j 0x00000011
40	20020001	addi $v0, $0, 0x0001
44	AC020054	sw $v0, 0x0054, $0
48	08000012	j 0x00000012

c:

jalsub:

IM[PC]

R[31] <- PC + 4

PC <- R[rs] - R[rt]

sra:

IM[PC]

R[rd] <- R[rt] >> shamt

PC -> PC +4

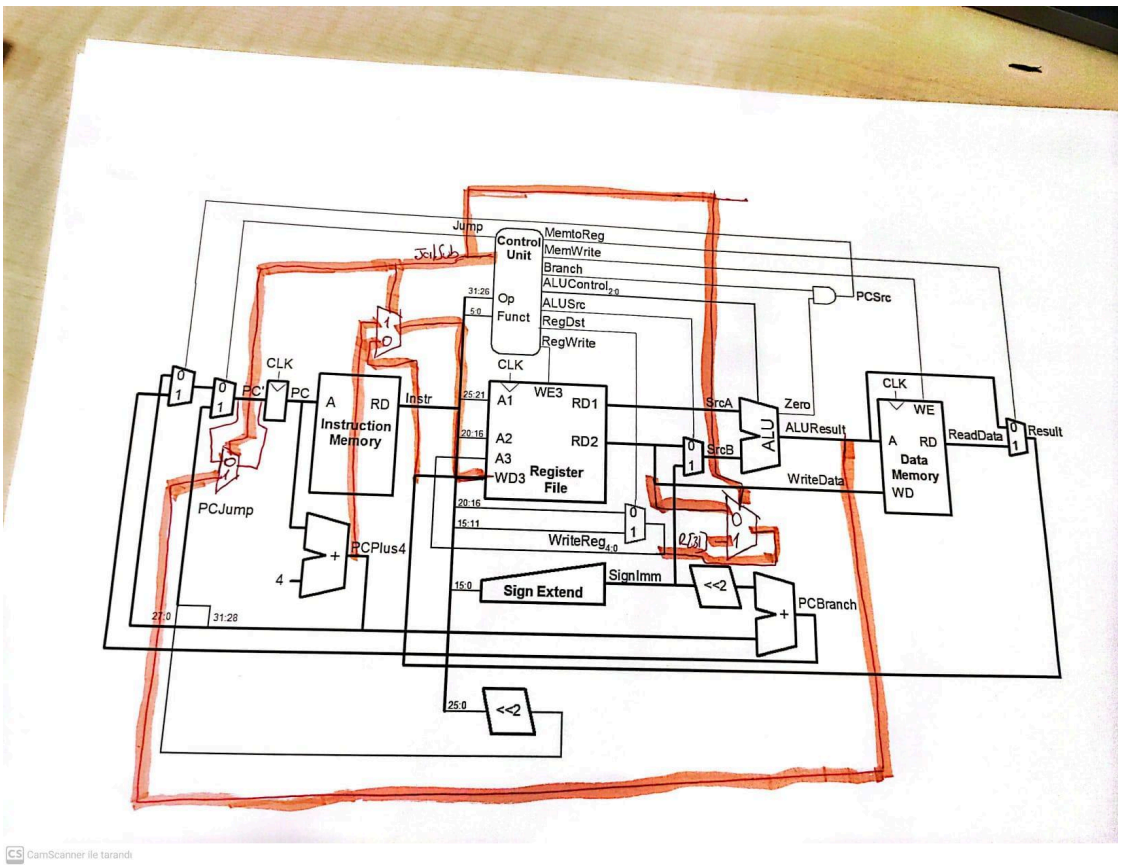
d:

jalsub:

Integrate an additional input into the PC to manage the calculation of R[rs] minus R[rt]. For the return address, direct the computed PC + 4 value to register $ra (register 31) and include a multiplexer to select between updating $ra or outputting the normal instruction result.

sra:

New control signal added to ALU.



e:

Instruction	Opcode	RegWrite	Reg Dst	ALU SRC	Branch	Mem Write	Mem to Reg	ALU OP	Jump	jalSub
R-type	0	1	1	0	0	0	0	10	0	0
lw	100011	1	0	1	0	0	1	00	0	0
sw	101011	0	x	1	0	1	x	00	0	0
beq	000100	0	x	0	1	0	x	01	0	0
addi	001000	1	0	1	0	0	0	00	0	0
j	000010	0	x	x	x	0	x	xx	1	0
jalsub	0	1	x	0	0	0	0	xx	1	1
sra	0	1	1	0	0	0	0	11	0	0

ALUOp	Funct	ALUControl	Operation
00	X	010	Add
01	X	110	Subtract
10	100000	010	Add
10	100010	110	Subtract
10	100100	000	And
10	100101	001	Or
10	101010	111	Set less than
10	000011	011	sra
10	1100110	110	jalsub

f:

```
.org 0
   addi $a0, $zero, 0x18
   addi $a1, $zero, 0x08
   jalsub $a0, $a1

   addi $a2, $zero, 3
   addi $a3, $zero, 6

.org 0x10
continue:
   add $t0, $a0, $a1
   sub $t1, $a0, $a1
   and $t2, $a0, $a1
   or  $t3, $a0, $a1
   slt $t4, $a1, $a0
   lui $t5, 0x1001
   ori $t6, $zero, 0xFFFF
   sra $t7, $t6, 4
   sw  $t0, 0($sp)
   lw  $s0, 0($sp)
   beq $a0, $a1, skip
   addi $s1, $zero, 1

skip:
   jr $ra
```

g:

```
`timescale 1ns / 1ps

module system_top (
    input logic clk,
    input logic rst,
    output logic [31:0] out_wdata,
    output logic [31:0] out_daddr,
    output logic [31:0] out_pc,
    output logic [31:0] out_inst,
    output logic [31:0] out_rdata,
    output logic out_mwr
);
    cpu_core core (clk, rst, out_pc, out_inst, out_mwr, out_daddr, out_wdata, out_rdata);
    inst_mem imem_inst (out_pc[7:0], out_inst);
    data_mem dmem_inst (clk, out_mwr, out_daddr, out_wdata, out_rdata);
endmodule

module data_mem (
    input logic clk,
    input logic we,
    input logic [31:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata
);
    logic [31:0] mem[63:0];
    assign rdata = mem[addr[31:2]];
    always_ff @(posedge clk)
        if (we)
            mem[addr[31:2]] <= wdata;
endmodule

module inst_mem (
    input logic [7:0] addr,
    output logic [31:0] inst
);
    logic [31:0] mem[63:0];
    initial begin
        mem[0] = 32'h00C50023;
        mem[1] = 32'h20020005;
        mem[2] = 32'h2003000c;
        mem[7] = 32'h2004000a;
    end
    assign inst = mem[addr[7:2]];
endmodule

module cpu_core (
```

```
    input logic clk,
    input logic rst,
    output logic [31:0] pc,
    input logic [31:0] inst,
    output logic mwr,
    output logic [31:0] alu_res,
    output logic [31:0] wdata,
    input logic [31:0] rdata
);
    logic mem2reg, pc_sel, src_sel, rd_sel, wr_en, jp;
    logic jsub;
    logic [2:0] alu_ctrl;
    logic zero_flag;
    ctrl_unit ctrl (inst[31:26], inst[5:0], zero_flag, mem2reg, mwr, pc_sel, src_sel, rd_sel,
wr_en, jp, alu_ctrl, jsub);
    dp_unit dp (clk, rst, mem2reg, pc_sel, src_sel, rd_sel, wr_en, jp, jsub, alu_ctrl, zero_flag,
pc, inst, alu_res, wdata, rdata);
endmodule

module ctrl_unit (
    input logic [5:0] op,
    input logic [5:0] func,
    input logic zero,
    output logic mem2reg,
    output logic mwr,
    output logic pc_sel,
    output logic src_sel,
    output logic rd_sel,
    output logic wr_en,
    output logic jp,
    output logic [2:0] alu_ctrl,
    output logic jsub
);
    logic [1:0] alu_op;
    logic br;
    main_decoder mdec (op, mem2reg, mwr, br, src_sel, rd_sel, wr_en, jp, alu_op);
    alu_decoder adec (func, alu_op, alu_ctrl);
    assign pc_sel = br & zero;
    assign jsub = (op == 6'b000000 && func == 6'b100011);
endmodule

module main_decoder (
    input logic [5:0] op,
    output logic mem2reg,
    output logic mwr,
    output logic br,
    output logic src_sel,
    output logic rd_sel,
    output logic wr_en,
    output logic jp,
```

```
    output logic [1:0] alu_op
);
    logic [8:0] signals;
    always_comb begin
        case (op)
            6'b000000: signals = 9'b110000100;
            6'b100011: signals = 9'b101001000;
            6'b101011: signals = 9'b001010000;
            6'b000100: signals = 9'b000100010;
            6'b001000: signals = 9'b101000000;
            6'b000010: signals = 9'b000000001;
            default:   signals = 9'bxxxxxxxxx;
        endcase
    end
    assign {wr_en, rd_sel, src_sel, br, mwr, mem2reg, alu_op, jp} = signals;
endmodule

module alu_decoder (
    input logic [5:0] func,
    input logic [1:0] alu_op,
    output logic [2:0] alu_ctrl
);
    always_comb begin
        case (alu_op)
            2'b00: alu_ctrl = 3'b010;
            2'b01: alu_ctrl = 3'b110;
            default: begin
                case (func)
                    6'b100000: alu_ctrl = 3'b010;
                    6'b100010: alu_ctrl = 3'b110;
                    6'b100011: alu_ctrl = 3'b110;
                    6'b100100: alu_ctrl = 3'b000;
                    6'b100101: alu_ctrl = 3'b001;
                    6'b101010: alu_ctrl = 3'b111;
                    default:   alu_ctrl = 3'bxxx;
                endcase
            end
        endcase
    end
endmodule

module dp_unit (
    input logic clk,
    input logic rst,
    input logic mem2reg,
    input logic pc_sel,
    input logic src_sel,
    input logic rd_sel,
    input logic wr_en,
    input logic jp,
```

```
    input logic jsub,
    input logic [2:0] alu_ctrl,
    output logic zero,
    output logic [31:0] pc,
    input logic [31:0] inst,
    output logic [31:0] alu_res,
    output logic [31:0] wdata,
    input logic [31:0] rdata
);
    logic [31:0] next_pc, pc_plus4, branch_addr, mux_out;
    logic [31:0] imm_ext, imm_shift, reg_A, reg_B;
    add_32 add1 (pc, 32'd4, pc_plus4);
    sign_extend se (inst[15:0], imm_ext);
    shift2 sh (imm_ext, imm_shift);
    add_32 add2 (pc_plus4, imm_shift, branch_addr);
    mux_2to1 #(32) mux1 (pc_plus4, branch_addr, pc_sel, mux_out);
    always_comb begin
        if (jsub)
            next_pc = alu_res;
        else if (jp)
            next_pc = {pc_plus4[31:28], inst[25:0], 2'b00};
        else
            next_pc = mux_out;
    end
    reg_flipflop #(32) reg_pc (clk, rst, next_pc, pc);
    wire [4:0] dest_reg;
    assign dest_reg = jsub ? 5'd31 : (rd_sel ? inst[15:11] : inst[20:16]);
    wire [31:0] write_val;
    assign write_val = jsub ? pc_plus4 : (mem2reg ? rdata : alu_res);
    register_file rf (clk, wr_en, inst[25:21], inst[20:16], dest_reg, write_val, reg_A, wdata);
    mux_2to1 #(32) mux2 (wdata, imm_ext, src_sel, reg_B);
    alu alu_inst (reg_A, reg_B, alu_ctrl, alu_res, zero);
endmodule

module register_file (
    input logic clk,
    input logic we,
    input logic [4:0] ra1,
    input logic [4:0] ra2,
    input logic [4:0] wa,
    input logic [31:0] wd,
    output logic [31:0] rd1,
    output logic [31:0] rd2
);
    logic [31:0] reg_arr[31:0];
    always_ff @(posedge clk)
        if (we)
            reg_arr[wa] <= wd;
    assign rd1 = (ra1 != 0) ? reg_arr[ra1] : 32'd0;
    assign rd2 = (ra2 != 0) ? reg_arr[ra2] : 32'd0;
```

```
endmodule

module alu (
   input logic [31:0] a,
   input logic [31:0] b,
   input logic [2:0] alu_ctrl,
   output logic [31:0] result,
   output logic zero
);
   always_comb begin
      case (alu_ctrl)
         3'b010: result = a + b;
         3'b110: result = a - b;
         3'b000: result = a & b;
         3'b001: result = a | b;
         3'b111: result = (a < b) ? 32'd1 : 32'd0;
         default: result = {32{1'bx}};
      endcase
   end
   assign zero = (result == 32'd0) ? 1'b1 : 1'b0;
endmodule

module add_32 (
   input logic [31:0] a,
   input logic [31:0] b,
   output logic [31:0] y
);
   assign y = a + b;
endmodule

module shift2 (
   input logic [31:0] a,
   output logic [31:0] y
);
   assign y = {a[29:0], 2'b00};
endmodule

module sign_extend (
   input logic [15:0] a,
   output logic [31:0] y
);
   assign y = {{16{a[15]}}, a};
endmodule

module reg_flipflop #(parameter WIDTH = 8) (
   input logic clk,
   input logic rst,
   input logic [WIDTH-1:0] d,
   output logic [WIDTH-1:0] q
);
```

```
    always_ff @(posedge clk, posedge rst)
        if (rst)
            q <= '0;
        else
            q <= d;
endmodule

module mux_2to1 #(parameter WIDTH = 8) (
    input logic [WIDTH-1:0] d0,
    input logic [WIDTH-1:0] d1,
    input logic s,
    output logic [WIDTH-1:0] y
);
    assign y = s ? d1 : d0;
endmodule


`timescale 1ns / 1ps



module testbench();
    logic clk, rst;
    logic [31:0] out_wdata, out_daddr, out_pc, out_inst, out_rdata;
    logic out_mwr;

    system_top dut (clk, rst, out_wdata, out_daddr, out_pc, out_inst, out_rdata, out_mwr);

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $monitor("Time=%0t | PC=%h | Inst=%h | $ra=%h", $time, out_pc, out_inst,
dut.core.dp.rf.reg_arr[31]);
    end

    initial begin
        $dumpfile("testbench.vcd");
        $dumpvars(0, testbench);

        rst = 1; #10;
        rst = 0;

        dut.imem_inst.mem[0] = 32'h00C50023;
        dut.core.dp.rf.reg_arr[6] = 32'h20;
        dut.core.dp.rf.reg_arr[5] = 32'h04;

        #20;
```

```
        $display("\n----- Final Results -----");
        $display("PC:    %h", out_pc);
        $display("$ra:   %h", dut.core.dp.rf.reg_arr[31]);

        $finish;
    end
endmodule
```